Report of Digital literacy

CSA2001: Fundamentals in AIML
Name of the Student: Dhanya Srivastava
Registration Number: 25BAI10791
Slot: B11+B12+B13
Semester: 2

1. Introduction

Problem Statement

Medical diagnosis is a difficult thinking process that involves using **a lot of knowledge** to make sense of a patient's symptoms. When conditions are not recognized quickly, especially in areas with limited resources, it can cause serious problems. This project creates a **rule-based medical diagnosis system using Prolog**. Patients can enter their symptoms, and the system uses an organized set of information to figure out possible health issues.

Motivation

Expert Systems were among the earliest and most impactful applications of **Artificial Intelligence**. Systems like **MYCIN (1970s, Stanford)** demonstrated that structured rule-based reasoning could rival human expert performance in specific domains. This project revisits that classical AI paradigm using modern tools

Objective

• Build a Prolog knowledge base of symptoms and diseases using **first-order logic**

• Implement a **backward-chaining** inference engine in Prolog

• Create a REST API backend (Python/Flask) that calls the Prolog engine

• Build an interactive web frontend for end users

• Demonstrate core CSA2001 concepts: knowledge representation, search, inference

2. Background and Literature

Expert Systems

An expert system is a computer program that uses AI techniques to simulate the **decision-making ability** of a human expert in a specific domain. It consists of two core components:

• **Knowledge Base:** A collection of facts and rules representing domain expertise

• **Inference Engine:** A reasoning mechanism that applies rules to facts to derive conclusions

Prolog for AI

Prolog (Programming in Logic) is a **declarative logic programming language** that is particularly well-suited for expert systems because:
* It natively supports backward chaining — starting from a goal and working backwards to find supporting facts
* Rules can directly encode expert knowledge in a **human-readable form**
* Prolog's **unification mechanism handles pattern matching automatically**

Relationship to Course Syllabus

CO	Topic	How This Project Covers It
CO2	Prolog Programming	Entire knowledge base written in Prolog
CO2	Knowledge Representation	Facts and rules in first-order logic
CO1	Intelligent Agents	System acts as a reasoning agent
CO4	Classification (ML)	Threshold-based symptom matching
CO5	Case Study	Applied AI in healthcare domain

3. System Architecture

```
┌──────────────────────────────────────────────────────────┐
│                      USER (Browser)                      │
│                frontend/index.html                       │
└──────────────────────────────────────────────────────────┘
                          │
                          │ HTTP POST /api/diagnose
                          ▼
┌──────────────────────────────────────────────────────────┐
│                Python Flask Backend                      │
│                backend/app.py                            │
│    - Receives symptom list                               │
│    - Calls SWI-Prolog subprocess                         │
│    - Parses and returns JSON results                     │
└──────────────────────────────────────────────────────────┘
                          │
                          │ subprocess call
                          ▼
┌──────────────────────────────────────────────────────────┐
│            SWI-Prolog Inference Engine                   │
│            prolog/knowledge_base.pl                      │
│    - 36 symptom facts                                    │
│    - 15 disease rules                                    │
│    - match_percentage/3 predicate                        │
│    - diagnose/6 predicate                                │
│    - get_all_diagnoses/2 predicate                       │
└──────────────────────────────────────────────────────────┘
```

4. Implementation

Three Layer Architecture

Layer	Technology	What it does
Frontend	HTML, CSS, JavaScript	User interface — shows symptoms, collects input, displays results
Backend	Python + Flask	Receives symptoms from browser, calls Prolog, returns results as JSON
AI Engine	SWI-Prolog	Applies rules to symptoms and produces diagnosis

5. Knowledge Base Design

What is Knowledge Base

A knowledge base is a **collection of facts and rules that represent domain expertise**. In this project, the knowledge base contains facts about symptoms and rules about diseases. It is stored in a file called **knowledge_base.pl**.

Symptom Facts

Each symptom is stored as a simple Prolog fact with two parts — **an ID and a description.** There are **36 symptoms** in total covering different body systems.

```
symptom(fever, 'High body temperature').
symptom(cough, 'Persistent coughing').
symptom(headache, 'Pain in the head or upper neck').
% ... 36 symptoms total
```

Disease Rules

Each disease is stored with **five pieces of information**: its name, the list of symptoms associated with it, a short description, its severity level, and a list of precautions.

```
disease(influenza,
    [fever, cough, sore_throat, body_ache, headache, fatigue, chills],
    'A contagious respiratory illness caused by influenza viruses.',
    moderate,
    ['Get annual flu vaccine', 'Rest adequately', 'Drink plenty of fluids']).
```

The Inference Engine

The inference engine has **three predicates** that work together to produce a diagnosis:
• **count_matching_symptoms** — goes through the disease's symptom list and counts how many the user reported.
• **match_percentage** — divides matched symptoms by total symptoms and multiplies by 100.
• **diagnose — succeeds** (returns a result) only if the match percentage is 50 or above

```
diagnose(UserSymptoms, Disease, Pct, Desc, Sev, Prec) :-
    disease(Disease, DiseaseSymptoms, Desc, Sev, Prec),
    match_percentage(DiseaseSymptoms, UserSymptoms, Pct),
    Pct >= 50.
```

Diseases in the Knowledge Base

Disease	No. of Symptoms	Severity
Common Cold	5	Mild
Influenza	7	Moderate
COVID-19	7	Moderate
Dengue Fever	7	Moderate
Malaria	7	Severe
Typhoid Fever	7	Moderate
Gastroenteritis	6	Mild
Pneumonia	6	Severe
Allergic Rhinitis	5	Mild
Type 2 Diabetes	6	Severe
Arthritis	5	Moderate
Meningitis	6	Severe
Jaundice	6	Moderate
Migraine	6	Moderate
Asthma	4	Moderate

How Prolog Reasons - Backward Chaining

Prolog uses a technique called **backward chaining** to answer questions. Instead of going through all diseases and checking each one (forward chaining), Prolog starts from the goal — 'what disease does this person have?' — and works backwards to find facts that support it.

Example: Suppose the user reports: fever, cough, fatigue, loss_of_smell, loss_of_taste
• Prolog tries the goal: diagnose([fever,cough,fatigue,loss_of_smell,loss_of_taste], Disease, ...).
• It picks the first disease rule — common_cold — and checks: how many of [runny_nose, sneezing, sore_throat, cough, headache] are in the user's list?
• Only 'cough' matches. That is 1 out of 5 = 20%. Less than 50%, so common_cold is rejected.
• It then tries influenza — fever, cough, fatigue match. 3 out of 7 = 43%. Rejected.
• It tries covid_19 — fever, cough, fatigue, loss_of_smell, loss_of_taste all match. 5 out of 7 = 71%. Accepted!
• Prolog **continues checking all remaining diseases and collects all that pass the 50% threshold.**

This process happens automatically — the programmer only needs to define the facts and rules. Prolog handles all the searching

6. Sample Queries and Result

Case 1 - Flu Symptoms

Input symptoms: fever, cough, body_ache, headache, fatigue, chills

Disease	Match %	Severity	Reason
Influenza	86%	Moderate	6 out of 7 symptoms matched
Pneumonia	67%	Severe	4 out of 6 symptoms matched
Malaria	57%	Severe	4 out of 7 symptoms matched

Case 2 - Covid 19 Symptoms

Input symptoms: fever, cough, fatigue, loss_of_smell, loss_of_taste

Disease	Match %	Severity	Reason
COVID-19	71%	Moderate	5 out of 7 symptoms matched
Influenza	57%	Moderate	4 out of 7 symptoms matched

Case 3 - Diabetes Indicator

Input symptoms: frequent_urination, excessive_thirst, fatigue, blurred_vision

Disease	Match %	Severity	Reason
Type 2 Diabetes	67%	Severe	4 out of 6 symptoms matched

7. Limitation

• **All symptoms are treated equally.** In real medicine, some symptoms are more important than others (e.g., stiff neck in meningitis is more critical than fatigue).
• The **50% threshold can cause false positives for diseases with very few symptoms.** For example, Asthma has only 4 symptoms — matching just 2 gives 50%.
• The system does not consider how long symptoms have been present, which matters medically.
• There are **no lab test inputs.** Real diagnosis often depends on blood tests, X-rays, etc.
• The knowledge base is **limited to 15 diseases**. Many other conditions are not covered.

8. Future Enhancement

• Add symptom weights so that **more distinctive symptoms carry higher importance in the score.**
• Use **Bayesian probability** to express confidence levels instead of simple percentages.
• Add a **conversational interface where the system asks follow-up questions.**
• Include more diseases and connect to real medical ontologies like **ICD-10.**
• **Allow patients to input lab values like blood sugar, hemoglobin etc. for better accuracy**

9. Conclusion

This project demonstrates how **Artificial Intelligence techniques — specifically knowledge representation and rule-based reasoning in Prolog** — can be applied to solve a real-world problem like medical diagnosis. The system successfully identifies potential diseases from user-reported symptoms using a threshold-based matching approach implemented entirely in **Prolog's backward chaining inference mechanism.**

The project directly covers multiple course outcomes of CSA2001, particularly CO2 (Prolog programming and knowledge representation), CO1 (intelligent agent design), and CO5 (applied AI case study). The three-tier architecture also demonstrates how classical AI can be integrated into modern web-based software.

10. References

• **Russell, S. and Norvig, P. (2009).** Artificial Intelligence: A Modern Approach, 3rd Edition. Prentice Hall. [Primary textbook for CSA2001]
• **Bratko, I. (2011).** Prolog: Programming for Artificial Intelligence, 4th Edition. Addison-Wesley.
• **SWI-Prolog Official Documentation. https://www.swi-prolog.org/pldoc/**
• **Flask Web Framework Documentation. https://flask.palletsprojects.com/**
• **Shortliffe, E.H. (1976). MYCIN:** Computer-Based Medical Consultations. Elsevier. [Original medical expert system paper]